차세대 웹 기반 3D 인테리어 디자인 플랫폼 구축을 위한 Zero-to-One 기술 분석 및 실행 전략 보고서

1. 서론: 프로토타입의 실패 원인 분석과 전략적 전환의 필요성

현재 사용자가 경험하고 있는 "작업을 수행했음에도 불구하고 체감상 변화가 없고 UI가 미려하지 않다"는 문제는 현대 웹 3D 개발, 특히 React와 Three.js를 결합한 복잡한 애플리케이션 개발 과정에서 빈번하게 발생하는 '기술적 정체 구간(Technical Plateau)'의 전형적인 사례이다. 사용자가 제공한 working.pdf와 에이전트들의 분석 보고서(3d공간 구축 웹사이트 개발 로드맵.pdf, Zero-to-One 3D 웹 인테리어 플랫폼 개발 가이드.pdf)를 종합적으로 심층 분석한 결과, 현재의 프로젝트는 기능적 구현 단계(Functional Implementation)에서 사용자 경험 최적화 단계(Experiential Optimization)로 넘어가지 못한 상태이며, 특히 핵심 코어인 AI 엔진과 렌더링 파이프라인에서 구조적인 결함이 발견되었다.

본 보고서는 단순한 코드 수정을 넘어, Dverso Studio나 Caffe.design과 같은 하이엔드(High-end) 웹 3D 플랫폼 수준의 완성도를 달성하기 위해 필요한 아키텍처, AI 모델링, 렌더링 파이프라인, 그리고 인터랙션 설계를 포괄하는 **'Zero-to-One'** 구축 전략을 제시한다. 이는 기존 프로젝트의 레거시(Legacy)를 선별적으로 수용하되, 핵심 엔진을 완전히 교체하는 '소프트 리셋(Soft Reset)' 방식을 기반으로 한다.

1.1 현재 시스템의 병리학적 진단

사용자의 불만은 크게 세 가지 축으로 요약될 수 있다: "변한 게 없다(기능 정체)", "UI가 구리다(시각적 품질 저하)", "도면 인식이 안 된다(AI 성능 실패)".

1. AI 도면 인식의 실패 (The AI Hallucination Issue):
   현재 시스템은 Google의 Gemini 모델을 사용하여 2D 도면 이미지를 3D 모델로 직접 변환하려는 시도를 했다. 그러나 분석 결과, Gemini 모델은 도면의 복잡한 기하학적 구조를 이해하지 못하고 **더미 데이터(Dummy Data)**를 반환하거나, 사용자의 입력과 무관한 6x4m의 작은 박스만을 생성하는 현상이 반복되었다.1 이는 AI에게 '기하학적 좌표'를 직접 생성하도록 강제했기 때문에 발생한 문제로, LLM(Large Language Model)의 특성상 시각적 정보를 정밀한 유클리드 좌표계로 변환하는 데에는 한계가 있음을 시사한다. 결과적으로 기능적인 진보가 없었기에 사용자는 "변한 게 없다"고 느끼게 된 것이다.
2. 렌더링 파이프라인의 부재 (The Rendering Gap):
   "UI가 구리다"는 평가는 디자인 감각의 부재라기보다는 물리 기반 렌더링(PBR: Physically Based Rendering) 파이프라인의 부재에서 기인한다. 현재 코드는 Three.js의 기본 BoxGeometry와 단순한 색상이 적용된 MeshStandardMaterial을 사용하고 있으며, 조명 또한 AmbientLight와 단일 DirectionalLight에 의존하고 있다.1 현실 세계의 물체는

환경광의 반사, 그림자의 부드러움(Soft Shadow), 재질의 거칠기(Roughness) 등에 의해 입체감이 결정되는데, 현재 시스템은 이러한 요소가 결여되어 있어 플라스틱 장난감과 같은 이질감을 준다.

3. 아키텍처의 한계:
   기존 프로젝트는 Three.js의 명령형(Imperative) 로직과 React의 선언형(Declarative) 로직이 혼재되어 있어, 상태 관리와 렌더링 루프 간의 동기화가 어렵다. 이는 30~40 FPS 수준의 낮은 프레임 레이트와 메모리 누수(Memory Leak)를 유발하며, 사용자 경험을 저해하는 근본적인 원인이 된다.1

1.2 전략적 제언: 리팩토링 vs. 제로 투 원(Zero-to-One)

사용자는 "기존 프로젝트 위에서 작업할지, 새로 만들지"를 고민하고 있다. 두 PDF 보고서와 연구 자료를 종합한 결과, 기존의 모노레포(Monorepo) 구조는 유지하되, 3D 및 AI 관련 핵심 모듈은 새로운 디렉토리에서 처음부터 재설계하는 '하이브리드 Zero-to-One' 전략이 최선이다.

- 유지할 것: Supabase 인증, 데이터베이스 스키마, 라우팅 구조 등 인프라스트럭처.
- 재구축할 것: 3D 씬 그래프(R3F 기반), AI 분석 파이프라인(JSON Schema 기반), UI 오버레이 시스템.

이 방식은 인프라 구축에 소요되는 시간을 절약하면서도, 사용자가 원하는 '완전한 변화'를 기술적으로 담보할 수 있는 유일한 해결책이다.

---

2. 하이엔드 3D 웹 아키텍처: React Three Fiber 생태계의 도입

Dverso Studio와 같은 수준의 플랫폼을 구축하기 위해서는 순수 Three.js가 아닌, React 생태계와 긴밀하게 통합된 **React Three Fiber (R3F)** 생태계를 전면적으로 도입해야 한다.

2.1 기술 스택의 재정의

계층 (Layer)	추천 기술 스택	선정 근거 및 기대 효과
Core Framework	**Next.js 14+ (App Router)**	초기 로딩 속도(FCP) 최적화 및 SEO 대응. 무거운 AI 연산을 서버리스 함수로 분리하여 클라이언트 부하 감소.[1]
3D Engine	**React Three Fiber (R3F)**	3D 객체를 React

		컴포넌트처럼 관리. 복잡한 씬 그래프를 선언적으로 구성하여 코드의 가독성과 재사용성 극대화.[2]
State Management	**Zustand**	Redux와 달리 렌더링 루프 내부에서 발생하는 고빈도 상태 변경(카메라 이동 등)을 'Transient Update'로 처리하여 리렌더링 없이 60FPS 유지.[3]
Environment	**@react-three/drei**	카메라 컨트롤, 환경 맵, 로더 등 검증된 유틸리티 라이브러리. 개발 속도 단축 및 안정성 확보.[3]
Post-processing	**@react-three/postprocessing**	Bloom, Depth of Field, Noise 등 영화 같은 시각 효과를 위한 후처리 파이프라인 구축.[3]
Physics	**Rapier.js (@react-three/rapier)**	WASM 기반의 고성능 물리 엔진. 1인칭 시점에서의 벽 충돌 방지 및 자연스러운 이동 구현.[4]
AI Vision	**Claude 3.5 Sonnet / GPT-4o**	복잡한 도면의 구조적 이해 및 JSON 데이터 추출에 특화된 최신 멀티모달 모델.[5]

2.2 디렉토리 구조 설계 (Domain-Driven Design)

새로운 프로젝트 구조는 도메인 주도 설계(DDD)를 반영하여 apps/web/src 내부에 다음과 같이 구성해야 한다.

```
src/
├── components/
│  ├── canvas/ # 3D R3F 컴포넌트 (Scene, Lights, Models)
│  ├── overlay/ # 3D 위에 뜨는 2D UI (Inventory, Panels)
│  ├── dom/ # 일반적인 HTML 컴포넌트 (Navbar, Dashboard)
│  └── layouts/ # 페이지 레이아웃
```

```
├── hooks/ # useKeyboard, useStore 등 커스텀 훅
├── lib/
│ ├── ai/ # Claude/GPT API 호출 및 파싱 로직
│ ├── geometry/ # 2D 데이터를 3D 메쉬로 변환하는 알고리즘
│ └── supabase/ # 백엔드 통신 로직
├── stores/ # Zustand 스토어 (editorStore, uiStore)
└── types/ # TypeScript 타입 정의 (FloorPlan, Furniture 등)
```

이 구조는 시각적 요소(Canvas)와 로직(Lib/Stores), 인터페이스(Overlay)를 명확히 분리하여 유지보수성을 높이고, 향후 기능 확장에 유연하게 대처할 수 있게 한다.

---

3. 핵심 AI 엔진: 도면 인식의 정확도 혁신 (Semantic Parsing Pipeline)

사용자가 겪은 가장 큰 실패 요인인 '도면 인식 및 3D 생성' 문제를 해결하기 위해, 기존의 'Image-to-Model' 방식을 버리고 **'Semantic Parsing to Procedural Generation'** 방식으로 전환해야 한다.

3.1 AI 모델 교체: Gemini에서 Claude 3.5 Sonnet/GPT-4o로

연구 데이터에 따르면, **Claude 3.5 Sonnet**은 시각적 추론(Visual Reasoning)과 복잡한 지시 사항 이행 능력에서 GPT-4o와 대등하거나 우수한 성능을 보여주며, 특히 코딩 및 구조화된 데이터 생성에서 강점을 보인다.[5] 반면 GPT-4o는 응답 속도와 다국어 처리에 강점이 있다.[5] 도면 분석은 속도보다는 '정확한 구조 추출'이 핵심이므로, **Claude 3.5 Sonnet**을 1순위로, GPT-4o를 2순위로 권장한다.

3.2 스키마 기반 프롬프트 엔지니어링 (Schema-Driven Prompting)

AI가 환각(Hallucination)을 일으키지 않도록 하려면, AI에게 "3D 모델을 만들어줘"라고 요청하는 대신 "이 이미지에서 벽과 문의 좌표를 추출하여 JSON으로 달라"고 요청해야 한다. 이를 위해 엄격한 **JSON** 스키마를 정의하여 프롬프트에 포함시켜야 한다.[8]

추천 프롬프트 구조:

> **System:** 당신은 전문 건축 도면 분석가입니다. 이미지를 분석하여 건축적 위상(Topology) 데이터를 추출하십시오.
>
> **Task:**
>
> 1. 모든 벽(Wall)의 시작점과 끝점 좌표를 추출하십시오. (이미지 좌상단 0,0 기준 픽셀 좌표)
> 2. 모든 개구부(문, 창문)의 위치, 너비, 종류를 식별하십시오.
> 3. 도면의 스케일 바(Scale Bar)나 치수선을 찾아 픽셀 대 미터 비율을

계산하십시오.

Output Format (JSON Only):

JSON

```
{
 "scale_ratio": 0.05, // 1 pixel = 0.05 meters
 "walls": [
  { "start": [x, y], "end": [x, y], "thickness": 0.2 },
  ...
 ],
 "openings": [
  { "type": "door", "wall_index": 0, "position_on_wall": 0.5, "width": 0.9 },
  ...
 ],
 "rooms": }
 ]
}
```

이러한 접근 방식은 AI가 시각적 노이즈(가구 그림, 텍스트 등)를 무시하고 구조적 데이터에만 집중하게 만든다.

3.3 'Human-in-the-Loop' 보정 인터페이스

아무리 뛰어난 AI라도 도면 인식 정확도가 100%가 될 수는 없다. 따라서 AI가 생성한 JSON 데이터를 3D로 변환하기 전에, 사용자가 **2D** 상에서 수정할 수 있는 단계를 반드시 거쳐야 한다.

1. **AI** 분석 요청**:** 사용자가 도면 업로드.
2. 초안 생성**:** AI가 JSON 데이터를 반환.
3. **2D** 오버레이 편집**:** 원본 이미지 위에 SVG나 Canvas로 추출된 벽과 문을 선으로 표시. 사용자가 드래그하여 잘못된 꼭짓점을 수정하거나 누락된 벽을 추가.
4. **3D** 생성 확정**:** 수정된 데이터를 기반으로 최종 3D 메쉬 생성.

이 과정은 사용자가 결과물에 대해 통제권을 갖게 함으로써, AI 오류로 인한 좌절감을 획기적으로 줄여준다.[1]

---

4. 절차적 3D 생성 및 최적화 (Procedural Generation

Engine)

확정된 JSON 데이터를 바탕으로 Three.js 상에서 3D 공간을 절차적으로 생성하는 로직은 다음과 같다.

4.1 벽체 생성 및 CSG 연산 (Constructive Solid Geometry)

단순히 육면체를 배치하는 것이 아니라, 문과 창문이 뚫린 벽체를 만들어야 한다. 이를 위해 **CSG(Constructive Solid Geometry)** 기술을 활용한다.

- 벽 메쉬 생성**:** 각 벽 데이터(start, end, thickness, height)를 기반으로 ExtrudeGeometry를 사용하여 벽을 생성한다.
- 개구부 처리**:** 문과 창문 위치에 해당하는 'Negative Mesh(빼기용 메쉬)'를 생성한다.
- **CSG** 연산**:** three-bvh-csg 라이브러리를 사용하여 [벽 메쉬] - [개구부 메쉬] 연산을 수행한다. 이를 통해 실제 구멍이 뚫린 벽체가 생성되어, 1인칭 시점에서 창문 밖을 보거나 문을 통과하는 것이 가능해진다.[1]

4.2 바닥 및 천장 생성

- 바닥**:** 각 방의 폐곡선(Closed Loop) 좌표를 추출하여 ShapeGeometry로 바닥면을 생성한다. 이때 텍스처(마루, 타일)가 반복되도록 UV 매핑을 적절히 조정해야 한다.
- 천장**:** 바닥과 동일한 지오메트리를 생성하되, 방의 높이만큼 Y축으로 이동시키고 법선 벡터(Normal Vector)를 뒤집어 아래를 향하게 한다. **탑뷰(Top-view) 모드에서는 천장을 자동으로 숨김 처리(visible = false)**하여 내부를 볼 수 있게 해야 한다.[1]

---

5. 시각적 완성도 극대화**:** 렌더링 파이프라인 **(The Visuals)**

사용자가 "구리다"고 느끼는 UI를 "트렌디"하게 바꾸기 위해, 물리 기반 렌더링(PBR)과 포스트 프로세싱을 적용한다.

5.1 조명 및 환경 설정 (Environment & Lighting)

기존의 AmbientLight는 공간을 평면적으로 만든다. 이를 개선하기 위해 **IBL (Image-Based Lighting)**을 도입한다.

- **HDRI** 환경 맵**:** @react-three/drei의 <Environment preset="apartment" /> 등을 사용하여 실제 공간의 빛 정보를 씬에 적용한다. 이는 금속 재질의 가구나 유리창에 사실적인 반사를 부여한다.[10]
- **Contact Shadows:** @react-three/drei의 <ContactShadows />를 사용하여 가구와 바닥이 만나는 지점에 진하고 부드러운 그림자를 생성한다. 이는 물체가 둥둥 떠 있는 느낌을 없애고 바닥에 밀착된 느낌(Grounding)을 준다.[1]

5.2 시네마틱 후처리 (Post-processing Pipeline)

@react-three/postprocessing 라이브러리를 사용하여 다음 효과들을 체인으로 연결한다.

1. **Bloom (빛 번짐):** 창문이나 조명 기구 주변에 은은한 빛 번짐 효과를 주어 공간의 분위기를 몽환적이고 고급스럽게 만든다.[1]
2. **SSAO (Screen Space Ambient Occlusion):** 벽 모서리나 가구 틈새에 음영을 추가하여 공간의 깊이감을 극대화한다.[1]
3. **Vignette (비네팅):** 화면 가장자리를 어둡게 하여 시선을 중앙으로 집중시킨다.
4. **Noise (필름 그레인):** 아주 미세한 노이즈를 추가하여 디지털 렌더링 특유의 인위적인 매끄러움을 없애고 사실적인 질감을 부여한다.

5.3 텍스처 및 재질 관리

무료 에셋 사이트(Poly Haven 등)에서 구한 PBR 텍스처(Diffuse, Normal, Roughness, AO Map)를 MeshStandardMaterial에 정확히 매핑해야 한다. 또한, 텍스처 로딩 속도를 최적화하기 위해 모든 텍스처를 **WebP** 포맷으로 변환하고 해상도를 2K(2048x2048) 이하로 제한하는 것이 좋다.[1]

---

6. 인터랙션 및 UX 설계: 듀얼 모드 시스템

사용자의 요구사항인 "3인칭 배치"와 "1인칭 탐험"을 모두 충족시키기 위해 듀얼 모드 컨트롤러**(Dual-Mode Controller)** 시스템을 구축한다.

6.1 모드 전환 메커니즘

하나의 캔버스 안에서 상태(State)에 따라 카메라와 컨트롤러를 교체한다.

- 탑뷰**(Top-view)** 모드 **(편집 모드):**
  - 카메라**:** OrthographicCamera (정사영 카메라)를 사용하여 원근감 왜곡 없이 정확한 가구 배치가 가능하게 한다.
  - 컨트롤**:** OrbitControls를 사용하되, 회전각(polarAngle)을 고정하여 수직에서만 내려다보게 제한한다.
  - 기능**:** 드래그 앤 드롭으로 가구 배치, 벽 재질 변경, 천장 숨김.
  - **UI:** 가구 라이브러리 사이드바, 속성 패널 표시.
- **1**인칭**(FPS)** 모드 **(탐험 모드):**
  - 카메라**:** PerspectiveCamera (원근 카메라)를 사용, 시야각(FOV) 75도 설정.
  - 컨트롤**:** PointerLockControls를 사용하여 마우스로 시점을 회전하고 WASD로 이동한다.[11]
  - 물리**: Rapier.js**의 RigidBody (Capsule 형태)를 플레이어에게 적용하여 벽 통과를 방지하고 바닥 높이 변화에 대응한다.
  - 상호작용**:** 화면 중앙에 Raycaster를 발사하여 문이나 조명 스위치에 커서가 올라가면 상호작용 아이콘을 표시한다 (마인크래프트 스타일).[1]
  - **UI:** 인벤토리 단축바(Hotbar), 크로스헤어만 표시.

6.2 자연스러운 전환 애니메이션

모드 전환 시 카메라가 순간 이동하면 사용자가 방향 감각을 잃을 수 있다. framer-motion이나 maath 라이브러리를 사용하여 카메라의 위치와 줌(Zoom) 상태를 1~1.5초에 걸쳐 부드럽게 보간(Interpolation)해야 한다. 예를 들어, 탑뷰에서 1인칭으로 전환 시 카메라가 하늘에서 현관문 앞으로 부드럽게 하강하며 줌인되는 연출을 적용한다.[1]

---

7. 구현 로드맵 및 실행 가이드

사용자가 즉시 실행할 수 있는 단계별 계획이다.

Phase 1: 기반 구축 및 렌더링 환경 조성 (Week 1)

1. 초기 설정: Next.js + R3F + Tailwind CSS + Zustand 프로젝트 생성.
2. 렌더링 셋업: Canvas에 <Environment>, <ContactShadows>, <EffectComposer>(Bloom, SSAO) 설정.
3. 결과: 빈 공간이지만 빛과 그림자가 사실적으로 표현되는 '무대' 완성.

Phase 2: AI 도면 분석 파이프라인 구축 (Week 2)

1. API 통합: Claude 3.5 Sonnet API 연동. JSON 스키마 기반 프롬프트 작성.
2. 2D 에디터: AI 결과를 시각화하고 수정할 수 있는 간단한 2D 오버레이 UI 구현.
3. 결과: 도면 이미지를 넣으면 정확한 벽/문 좌표 데이터(JSON)를 얻을 수 있는 시스템.

Phase 3: 3D 절차적 생성 및 편집 (Week 3)

1. 3D 생성: JSON 데이터를 읽어 벽(CSG 적용), 바닥, 천장 메쉬 자동 생성.
2. 탑뷰 에디터: 가구(GLB 모델) 드래그 앤 드롭 배치 기능 구현.
3. 결과: 도면을 3D로 변환하고 가구를 배치할 수 있는 편집 도구 완성.

Phase 4: 1인칭 인터랙션 및 최적화 (Week 4)

1. FPS 모드: Rapier.js 물리 엔진 적용, PointerLockControls 구현, 문 열기/닫기 상호작용.
2. UI 폴리싱: 랜딩 페이지, 로딩 화면(Suspense), 모달 창 디자인 개선 (Glassmorphism 적용).
3. 최적화: InstancedMesh 적용, 텍스처 압축, React.memo 적용.
4. 결과: 마인크래프트처럼 걸어 다니며 상호작용 가능한 완성형 플랫폼.

---

8. 결론

"Plan2Space" 프로젝트의 성공은 기술 스택의 단순한 교체가 아닌, 사용자 경험(UX) 중심의 아키텍처 재설계에 달려 있다. 기존의 실패는 도구의 문제가 아니라 도구를 사용하는 방식의 문제였다.

본 보고서가 제안하는 **Zero-to-One** 전략은 다음 세 가지 핵심 가치를 제공한다:

1. **AI 신뢰성 회복:** 시맨틱 파싱과 휴먼 보정 단계를 통해 '사용 가능한' 3D 모델을 보장한다.
2. 시각적 압도감: PBR과 후처리를 통해 웹에서도 콘솔 게임 수준의 그래픽을 구현한다.
3. 몰입형 경험: 심리스한 모드 전환과 물리 기반 인터랙션을 통해 단순한 뷰어가 아닌 '공간 경험'을 제공한다.

지금 바로 새로운 디렉토리를 생성하고, R3F 기반의 새로운 캔버스를 띄우는 것부터 시작하기를 강력히 권고한다. 이것이 가장 빠르고 확실하게 "완성형"에 도달하는 길이다.

Works cited

1. Zero-to-One 3D 웹 인테리어 플랫폼 개발 가이드.pdf
2. Introduction to React Three Fiber - GeekyAnts, accessed December 30, 2025, https://geekyants.com/blog/introduction-to-react-three-fiber
3. React Three Fiber: Introduction, accessed December 30, 2025, https://r3f.docs.pmnd.rs/
4. Character controller - Rapier physics engine, accessed December 30, 2025, https://rapier.rs/docs/user_guides/javascript/character_controller/
5. Claude 3.5 Sonnet June Version vs GPT-4o - PromptLayer Blog, accessed December 30, 2025, https://blog.promptlayer.com/claude-3-5-sonnet-june-version-vs-gpt-4o/
6. Claude 3.5 Sonnet vs GPT 4o: Model Comparison 2025 - Galileo AI, accessed December 30, 2025, https://galileo.ai/blog/claude-3-5-sonnet-vs-gpt-4o-enterprise-ai-model-comparison
7. Claude 3.5 Sonnet vs GPT-4o: Complete AI Model Comparison - SentiSight.ai, accessed December 30, 2025, https://www.sentisight.ai/claude-3-5-sonnet-vs-gpt-4o-ultimate-comparison/
8. Mastering JSON Prompting for LLMs - MachineLearningMastery.com, accessed December 30, 2025, https://machinelearningmastery.com/mastering-json-prompting-for-llms/
9. Introduction to Schema Based Prompting: Structured inputs for Predictable outputs, accessed December 30, 2025, https://opper.ai/blog/schema-based-prompting
10. React-Three-Fiber: Enhancing Scene Quality with Drei + Performance Tips - Medium, accessed December 30, 2025, https://medium.com/@ertugrulyaman99/react-three-fiber-enhancing-scene-quality-with-drei-performance-tips-976ba3fba67a
11. why-agency/r3f-firstpersondragcontrols: A pointer-capture-free first person control component for React Three Fiber, using Cannon physics. - GitHub, accessed December 30, 2025, https://github.com/why-agency/r3f-firstpersondragcontrols
12. Controls - React Three Drei, accessed December 30, 2025, https://drei.docs.pmnd.rs/controls/introduction